FINAL PROJECT REPORT

Kurdish Data Explorer

An Interactive Topic-Modeling and Corpus Research Workspace for Kurdish Text

Project profile

Student name	**Sawab Hussein**
Student ID	**A25100071**
Course	**Project Based Course I**
Instructor	**Dr. Dara Sherwani**
Department	**Department of Computer Science**
Submission date	**July 24, 2026**

Open the live system

https://kdx-explorer.fly.dev

Source code: github.com/SawabS/NLP_KurdishDataExplorer

Abstract

Kurdish Data Explorer is a deployed, end-to-end artificial intelligence system for turning a large, unlabelled Kurdish text corpus into navigable semantic structure. The production corpus was gathered through the companion Bahdini Crawler, normalized and sampled into 100,000 paragraphs, embedded with either OpenAI `text-embedding-3-small` or NVIDIA `nemotron-3-embed-1b`, and modeled with BERTopic. The resulting topic hierarchy, semantic map, document review tools, search, and grounded question answering are delivered through a React single-page application and FastAPI service on Fly.io. On the identical sample, NVIDIA produced 32 topics with 99.448% coverage and NPMI coherence of 0.0491 in 337.4 seconds; OpenAI produced 38 topics with 99.132% coverage and NPMI of 0.0374 in 3,628.7 seconds. The report documents the as-built system, evaluates its practical trade-offs, and identifies corpus review, dialect-aware normalization, and controlled human evaluation as the highest-priority next steps.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Background and problem

Kurdish natural-language processing is constrained less by the absence of digital text than by the difficulty of turning scattered, inconsistent material into a usable research resource. Kurdish dialects use multiple orthographies; Arabic-script text contains variant code points; and morphologically rich word forms weaken simple term-frequency methods (Ahmadi, 2020; Azzat et al., 2023). The practical consequence is an accessibility gap: a large corpus may exist, yet a student or researcher cannot easily see its major themes, inspect representative passages, or trace a model output back to evidence.

This project addresses that gap with a working software system rather than a purely theoretical model. It combines corpus collection, normalization, transformer embeddings, density-based topic discovery, interactive visualization, document review, and retrieval-augmented generation (RAG). BERTopic was selected because it separates semantic representation, dimensionality reduction, clustering, and interpretable topic words, a design that has proved useful for short text in morphologically rich, low-resource languages (Grootendorst, 2022; Medvecki et al., 2024).

1.2 Objective and scope

The objective was to build and deploy a reproducible workspace that lets users discover, compare, and verify semantic structure in Kurdish corpora. The implemented scope includes file profiling; paragraph-level ingestion; normalization; two hosted embedding providers and two local research options; BERTopic fitting; topic hierarchies; semantic maps; document inspection and review; semantic search; grounded Q&A; artifact persistence; and a production web deployment. The live demonstration deliberately uses one fixed, unlabelled corpus and two fitted embedding runs so that users compare model behavior on identical evidence. It does not claim exhaustive linguistic coverage, supervised classification accuracy, or a publication-ready corpus.

2 Dataset Description

2.1 Source and collection

The live data originated in the author's Bahdini Crawler repository (Hussein, 2026). That project records three collection routes: a breadth-first crawler for public Kurdish websites, a Playwright collector for an electronic-library Facebook group, and a resumable Telethon downloader for three public book channels. Its manifests preserve source URLs, page/message identifiers, local file names, sizes, and extraction outcomes. Across the wider collection, the crawler reports 4,485 web documents (about 7.4 GB), 1,318 Facebook PDFs (about 6 GB), and 1,925 Telegram documents (about 12 GB). Embedded text is extracted with PyMuPDF and normalized with KLPT; scanned or suspect pages are routed to OCR and remain explicitly marked *unreviewed*.

2.2 Deployed analytical sample

The deployed source file, `corpus_unreviewed.txt`, is 231,912,193 bytes. Ingestion treats each paragraph containing at least three words as one document. It detected 376,292 eligible paragraphs and, using seed 42, selected 100,000 for embedding. The resulting table contains a stable document identifier, source name, text, topic assignment, and two-dimensional coordinates; it has no ground-truth class label. This is therefore an unsupervised topic-discovery dataset, not a classification benchmark.

Preprocessing performs Unicode and Kurdish-script cleanup through KLPT, removes empty or undersized records, and preserves the paragraph as the unit of analysis. The model vectorizer retains Unicode word tokens of at least three characters and removes Kurdish stop words. A key limitation is that the current

Table 1: Production dataset profile recorded in both deployed run manifests.

Property	Verified value
Language focus	Bahdini (Kurmanji) Kurdish, primarily Arabic script
Source form	OCR/native-text corpus compiled by Bahdini Crawler
Eligible documents	376,292 paragraphs ($\geq$ 3 words)
Embedded sample	100,000 paragraphs, seeded random sample
Labels/classes	None; topics are discovered, not supplied
Normalization	KLPT Sorani normalization applied
Production models	OpenAI `text-embedding-3-small`; NVIDIA `nemotron-3-embed-1b`

normalizer is configured for Sorani while the live corpus is Bahdini. This choice standardizes many shared Arabic-script variants but may alter dialect-specific forms; the original source text and review status must therefore remain available.

3 Methodology

3.1 Model selection

BERTopic was chosen over a fixed-topic classifier because the deployed corpus is unlabelled and users need interpretable, evidence-linked exploration. Its pipeline combines dense semantic embeddings with UMAP neighborhood preservation (McInnes et al., 2018), HDBSCAN density clustering (Campello et al., 2013), and class-based TF–IDF topic words. Unlike LDA, the number of topics does not have to be supplied in advance, and documents that do not belong to a dense region can remain outliers. MMR keyword diversification reduces near-duplicate descriptors.

Two hosted embedding models were fitted on exactly the same 100,000 documents. OpenAI `text-embedding-3-small` provides a broadly capable commercial baseline (OpenAI, 2024). NVIDIA `nemotron-3-embed-1b` provides 2,048-dimensional multilingual embeddings, a 32k-token context window, batching, and concurrent requests (NVIDIA, 2025). Keeping both fitted artifacts makes the model choice visible rather than hiding it behind a single score. For offline research, the repository also contains base multilingual MiniLM and a Sorani-adapted MiniLM trained with the unsupervised TSDAE objective (Wang et al., 2021).

3.2 Development and fitting process

The implementation is Python 3.11 with pandas, NumPy, PyArrow, KLPT, sentence-transformers, UMAP, HDBSCAN, BERTopic, scikit-learn, FastAPI, and Uvicorn. The client is React 18 and TypeScript with Vite, TanStack Query, Plotly, and deck.gl. Figure 1 summarizes the actual data path.

Each embedding adapter normalizes vectors to unit length and caches the complete matrix by a content-derived key. OpenAI requests are token-aware, chunk long documents, and use rate-limit-aware retries. NVIDIA requests use batches of 64, up to eight concurrent workers, server-side truncation at the end, and exponential retry for rate or server errors. UMAP reduces embeddings to five dimensions for HDBSCAN and separately to two dimensions for display. HDBSCAN discovers dense regions; BERTopic then reassigns eligible outliers by c-TF-IDF similarity, extracts ten words per topic, constructs a binary topic hierarchy, and optionally generates readable labels. NPMI is computed from topic-word co-occurrence. All outputs are written to an isolated `artifacts/<source>__<model>/` directory.

3.2.1 Training, tuning, and reproducibility

The production experiment is transductive and unsupervised: UMAP and HDBSCAN are fitted on the corpus that users explore, so a conventional supervised train/validation/test split is neither used nor

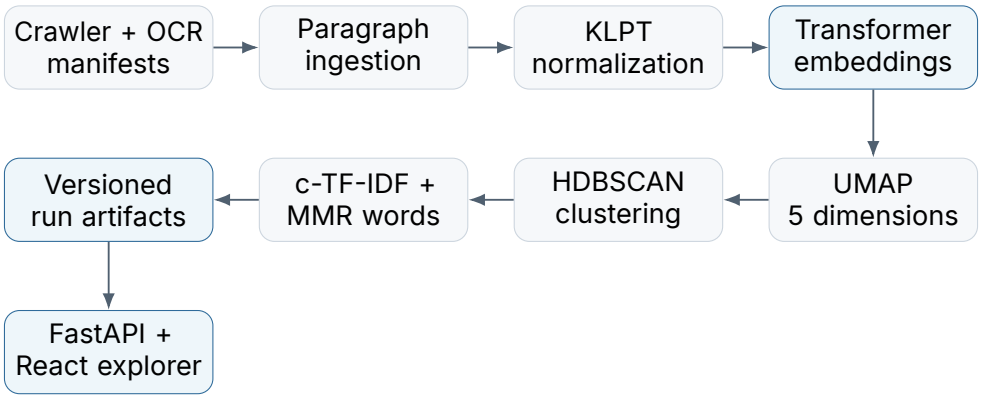


Figure 1: As-built corpus-to-interface pipeline. Fitting is performed once; ordinary exploration reads saved artifacts.

presented. Reproducibility instead comes from a fixed sample, seed 42, explicit provider identifiers, a content-keyed embedding cache, saved hyperparameters, and a complete `meta.json` manifest for every run. The tuning script can sweep HDBSCAN's minimum cluster size while reporting topic count, outlier rate, NPMI, topic diversity, and—when labels exist—normalized mutual information.

The optional local KDX MiniLM experiment supplies the report's true weight-training stage. It deduplicates KNDH headlines and sentence-split AsoSoft text, shuffles with seed 42, caps the corpus at 120,000 sentences, and trains a transformer denoising auto-encoder for one epoch. Each input sentence is corrupted and the decoder reconstructs it from the encoder representation; no class labels are required. The resulting sentence-transformer directory is versioned as a selectable offline model. Its observed anisotropic embedding space required a wider 50-neighbor UMAP graph, HDBSCAN leaf selection, and a minimum cluster size of 100 rather than silently reusing unsuitable defaults.

After clustering, topic naming is a separate, auditable enrichment step. Representative keywords and passages are sent to a configured chat model, while raw c-TF-IDF labels remain available if generation fails. FastAPI exposes saved runs through typed endpoints, and a single in-process worker serializes new fitting jobs to prevent competing memory-intensive pipelines. Thus model development, inference, labeling, and presentation remain separable and can be re-run or replaced independently.

3.3 Hyperparameters

4 Results and Evaluation

4.1 Production comparison

The comparison is controlled at the corpus level: both providers received the same sampled documents and downstream BERTopic configuration. Table 3 reports the persisted measurements. NVIDIA achieved higher NPMI, fewer outliers, and a 10.8× shorter end-to-end refit. OpenAI discovered six more topics, which may expose finer distinctions but also produced more unclustered documents. Because fit time includes cache state, provider calls, UMAP, clustering, labeling, and artifact writing, it is an operational benchmark rather than a pure embedding-throughput test.

4.2 Quality evidence and interpretation

NPMI measures whether a topic's highest-ranked words occur together more often than chance; positive values are preferred. Both production runs are positive, and NVIDIA's 0.04915 exceeds OpenAI's 0.03742 by about 31% relative. Coverage is excellent for both systems: 99,448 and 99,132 documents respectively were assigned to a topic. The NVIDIA structure has a larger leading topic (about 15%) than OpenAI (about

Table 2: Key model and training hyperparameters implemented in the repository.

Stage	Parameter	Value
Sampling	Random seed; documents	42; 100,000
UMAP (clustering)	Neighbors; components; minimum distance; metric	15; 5; 0.0; cosine
HDBSCAN	Minimum cluster size; minimum samples; metric	250; 10; Euclidean
Topic representation	c-TF-IDF; n-grams; topic words; MMR diversity	reduced frequent words; unigrams; 10; 0.3
Map projection	Components; minimum distance; metric	2; 0.1; cosine
NVIDIA adapter	Batch; workers; retries; truncation	64; 8; 6; end
Optional TSDAE	Sentences; epochs; batch; learning rate	120,000; 1; 16; 3×10^{-5}
Optional TSDAE	Scheduler; weight decay; optimizer	constant; 0; AdamW (sentence-transformers default)

Table 3: Results on the identical 100,000-document deployed sample.

Embedding run	Topics	Outliers	Coverage	NPMI
NVIDIA Nemotron-3-Embed-1B	32	552	99.448%	**0.04915**
OpenAI text-embedding-3-small	38	868	99.132%	0.03742

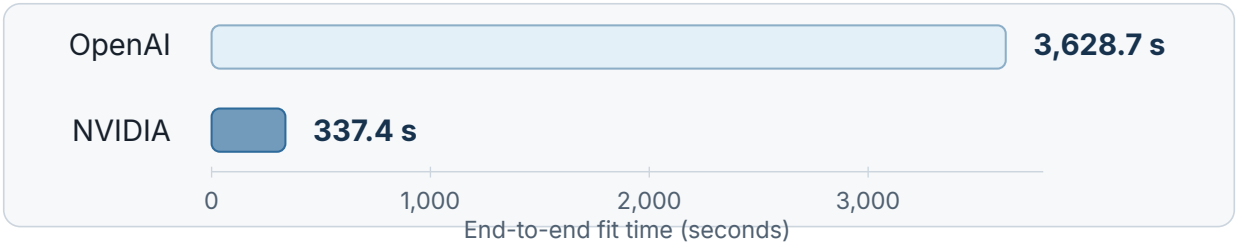


Figure 2: Measured end-to-end fitting time on the identical sample. NVIDIA completed the saved refit approximately 10.8 times faster than OpenAI.

11%), so its higher coherence and speed come with somewhat coarser organization.

An additional labelled KNDH experiment provides a useful development check. The Sorani-adapted KDX MiniLM improved BERTopic NPMI from −0.0470 for base MiniLM to +0.0567 on 50,000 headlines. In the same saved run, NMF scored 0.1071 and LDA −0.1490. These scores should not be mixed with the live Bahdini comparison because they use a different corpus; they demonstrate that representation choice and corpus-specific adaptation materially affect topic quality.

The interface supplied a second layer of practical evaluation. Automated browser verification confirmed that topic counts and coverage update when the embedding selector changes; the structure view opens representative documents; the map renders a bounded 12,000-point sample; the document table paginates, searches, and retains review states; and Insights reproduces the ingest manifest. A live RAG trial asked, "What are the main themes in this corpus?" The system retrieved eight passages and explicitly stated that the evidence was too heterogeneous for a definitive answer before offering a cautious language-and-culture interpretation. This is preferable to an unsupported summary and demonstrates that the evidence-only prompt can abstain, although one query is not a substitute for a formal retrieval benchmark.

A confusion matrix is not applicable: the deployed data has no target classes, and topic identifiers

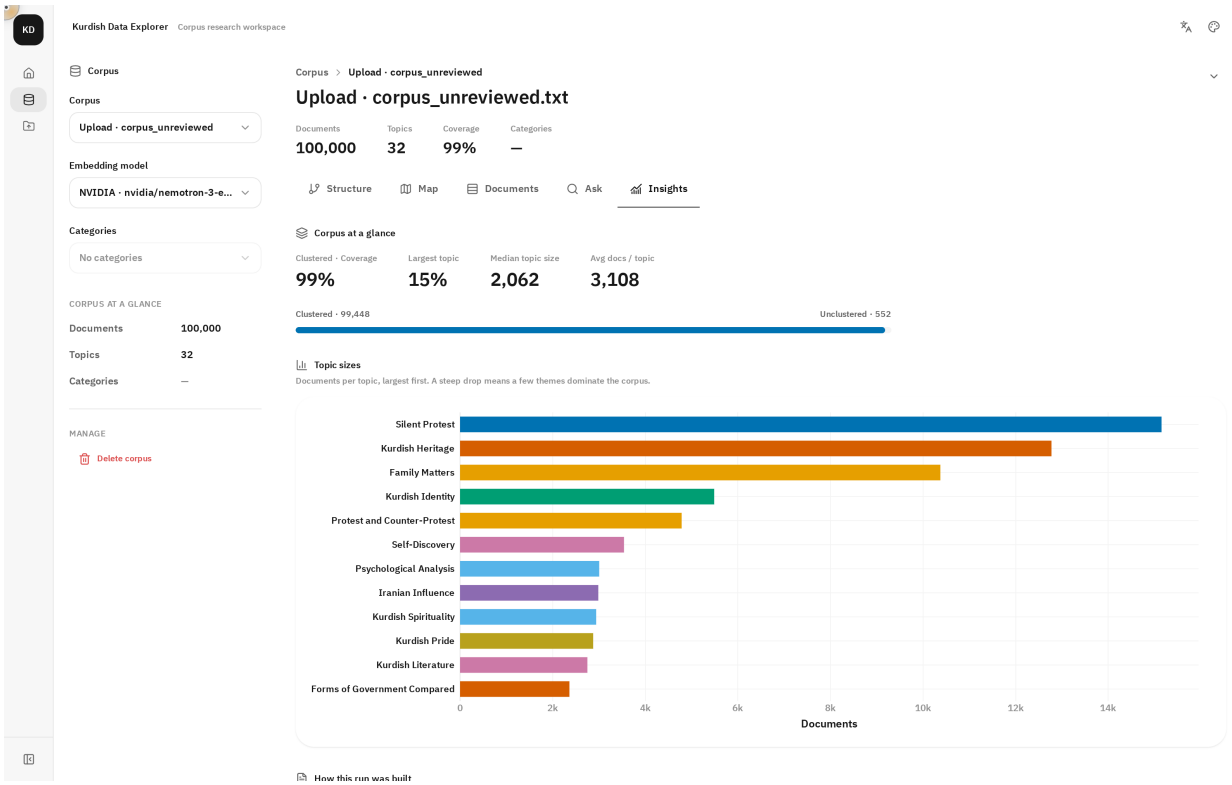


Figure 3: Deployed Insights workspace for NVIDIA: 32 topics, 99% rounded coverage, and topic-size distribution. Captured from the live application on July 24, 2026.

are arbitrary clusters rather than predicted class labels. Substituting a confusion matrix would imply supervision that does not exist. Appropriate evidence is therefore NPMI, outlier coverage, topic-size balance, cross-model comparison, and qualitative inspection of defining terms and representative documents.

4.3 Observed limitations

The evaluation reveals four practical limitations. First, corpus and OCR outputs are explicitly unreviewed; repeated headers, dictionaries, bibliographic pages, mixed dialects, or OCR errors can form topics. Second, automated topic labels are generated by an LLM and may overstate what the representative documents support. Third, two-dimensional UMAP is an exploratory projection: distance and apparent cluster area should not be interpreted as exact semantic geometry. Fourth, NPMI rewards word co-occurrence but does not measure usefulness to a Kurdish-speaking researcher. Human topic-intrusion and label-quality studies are still required.

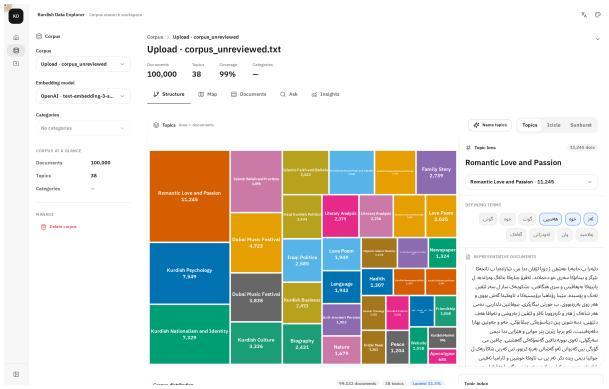
5 Deployment

5.1 Architecture and platform

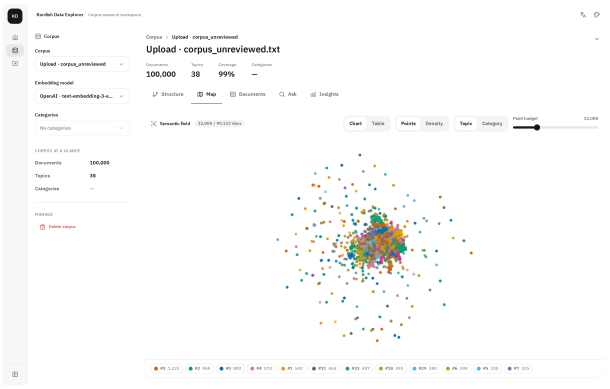
The system is deployed at https://kdx-explorer.fly.dev on Fly.io. A two-stage Docker build uses Node 20 to compile the React/Vite interface and Python 3.11 to run the FastAPI application. One Uvicorn process serves both the static single-page application and the `/api` routes on port 8080. The machine is a `shared-cpu-2x` instance with 2 GB RAM, HTTPS enforcement, and one always-running machine. The image includes only the approved `corpus-unreviewed` OpenAI and NVIDIA artifacts and their exact embedding caches. An mtime-keyed application cache loads one run at a time, which avoids holding both approximately 586 MB and 782 MB embedding matrices in limited memory.

5.2 User interaction

Users begin at a corpus overview and select a fitted embedding model. The **Structure** workspace presents the topic hierarchy, terms, and representative documents. **Map** shows a sampled 2D semantic field with topic or density encoding. **Documents** provides text search, topic filters, review states, nearest documents, and CSV export. **Ask** embeds an English or Kurdish query, retrieves eight nearest passages, and asks the configured chat model to answer only from those passages with numbered citations. **Insights** reports topic sizes, coverage, and exact ingest provenance. An upload workflow profiles TXT, CSV, TSV, Excel, or Parquet files before submitting a serialized background fitting job.



(a) Topic structure and evidence panel.



(b) Interactive semantic map.

Figure 4: Two primary views in the deployed application. Full-resolution captures are provided in Appendix B.

Production verification on July 24, 2026 returned HTTP 200 and `{"status":"ok"}` from `/api/health`; the source endpoint reported 100,000 documents and both fitted models. Browser automation opened the deployment, switched models, navigated every workspace, submitted a grounded question, and recorded no console or page errors during the capture session.

The deployment remains intentionally reproducible rather than elastic. Fitted artifacts are baked into the image, while API keys are supplied as runtime secrets and never committed. The health and source endpoints support release checks; HTTPS is forced at the platform boundary. A manual prepared-checkout deployment is currently necessary because the large corpus, embedding caches, and vendored design-system source are excluded from a clean public clone.

6 Conclusion and Future Improvements

Kurdish Data Explorer demonstrates a complete applied-AI workflow: source collection with provenance, corpus preparation, transformer embedding, unsupervised topic discovery, measurable evaluation, artifact persistence, a usable research interface, RAG-based evidence access, and public deployment. The fixed 100,000-document comparison makes the main result concrete. NVIDIA offered the strongest operational trade-off in this run—higher NPMI, fewer outliers, and substantially lower fit time—while OpenAI exposed a finer 38-topic structure. The result is not simply a model notebook; it is an inspectable system in which users can trace topics and answers back to text.

The next development phase should prioritize data quality over adding another model. OCR outputs should pass document-level human review, duplicates and boilerplate should be removed, licenses and source permissions should be audited, and Bahdini-specific normalization should replace the current Sorani-oriented configuration. Evaluation should add Kurdish-speaker topic intrusion, word intrusion, label correctness, and task-completion studies. Technically, durable job storage, approximate-nearest-neighbor search, streaming artifact loading, provider-cost logging, and role-based review would support larger corpora and collaborative use. Finally, a held-out, manually labelled Bahdini evaluation set would enable

NMI and controlled downstream retrieval tests without misrepresenting unsupervised topics as classes.

References

Ahmadi, S. (2020). Klpt—kurdish language processing toolkit. *Proceedings of the Second Workshop for NLP Open Source Software*, 72–84. https://doi.org/10.18653/v1/2020.nlposs-1.11

Azzat, M., Jacksi, K., & Ali, I. (2023). The kurdish language corpus: State of the art. *Science Journal of University of Zakho*, *11*(1), 125–131. https://doi.org/10.25271/sjuoz.2023.11.1.1123

Campello, R. J. G. B., Moulavi, D., & Sander, J. (2013). Density-based clustering based on hierarchical density estimates. *Advances in Knowledge Discovery and Data Mining*, 160–172. https://doi.org/10.1007/978-3-642-37456-2_14

Grootendorst, M. (2022). Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv*. https://doi.org/10.48550/arXiv.2203.05794

Hussein, S. (2026). *Bahdini crawler* (Version b346e5a94c29). Retrieved July 24, 2026, from https://github.com/SawabS/Bahdini_Crawler

McInnes, L., Healy, J., & Melville, J. (2018). Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv*. https://doi.org/10.48550/arXiv.1802.03426

Medvecki, D., Bašaragin, B., Ljajić, A., & Milošević, N. (2024). Multilingual transformer and bertopic for short text topic modeling: The case of serbian. *arXiv*. https://doi.org/10.48550/arXiv.2402.03067

NVIDIA. (2025). *Llama nemotron embed 1b v2*. Retrieved July 24, 2026, from https://build.nvidia.com/nvidia/llama-nemotron-embed-1b-v2/modelcard

OpenAI. (2024). *New embedding models and api updates*. Retrieved July 24, 2026, from https://openai.com/index/new-embedding-models-and-api-updates/

Wang, K., Reimers, N., & Gurevych, I. (2021). Tsdae: Using transformer-based sequential denoising auto-encoder for unsupervised sentence embedding learning. *Findings of the Association for Computational Linguistics: EMNLP 2021*, 671–688. https://doi.org/10.18653/v1/2021.findings-emnlp.59

Appendix

Appendix A: System Architecture Diagrams

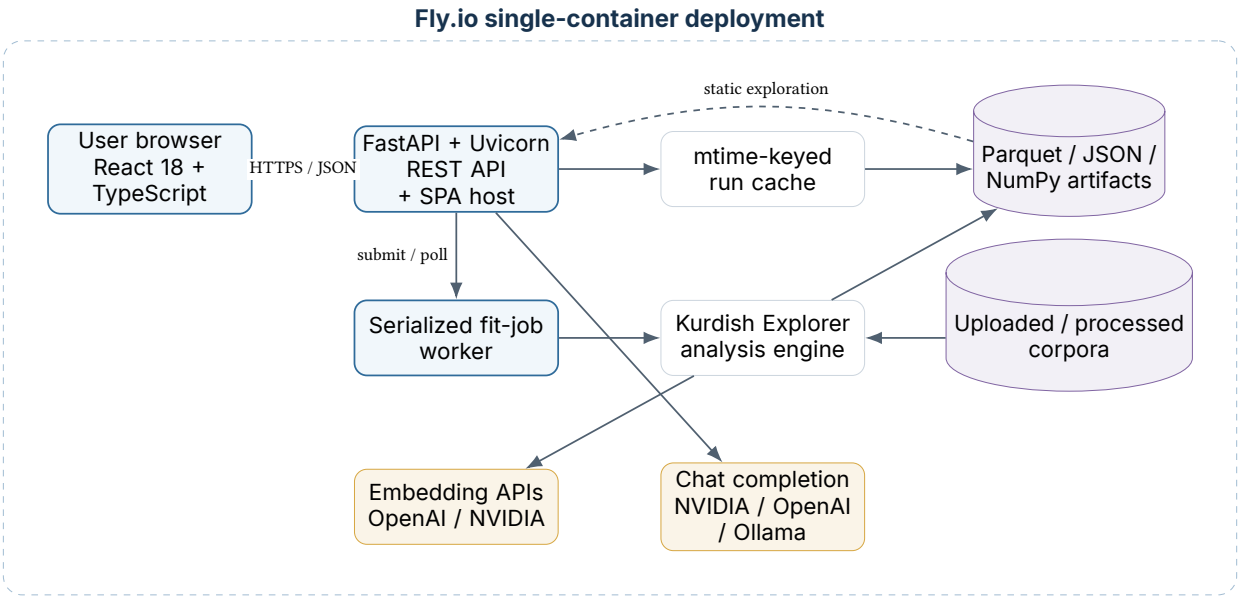


Figure 5: Production component architecture. Expensive fitting uses a serialized worker; ordinary browsing reads immutable run artifacts through the cache.

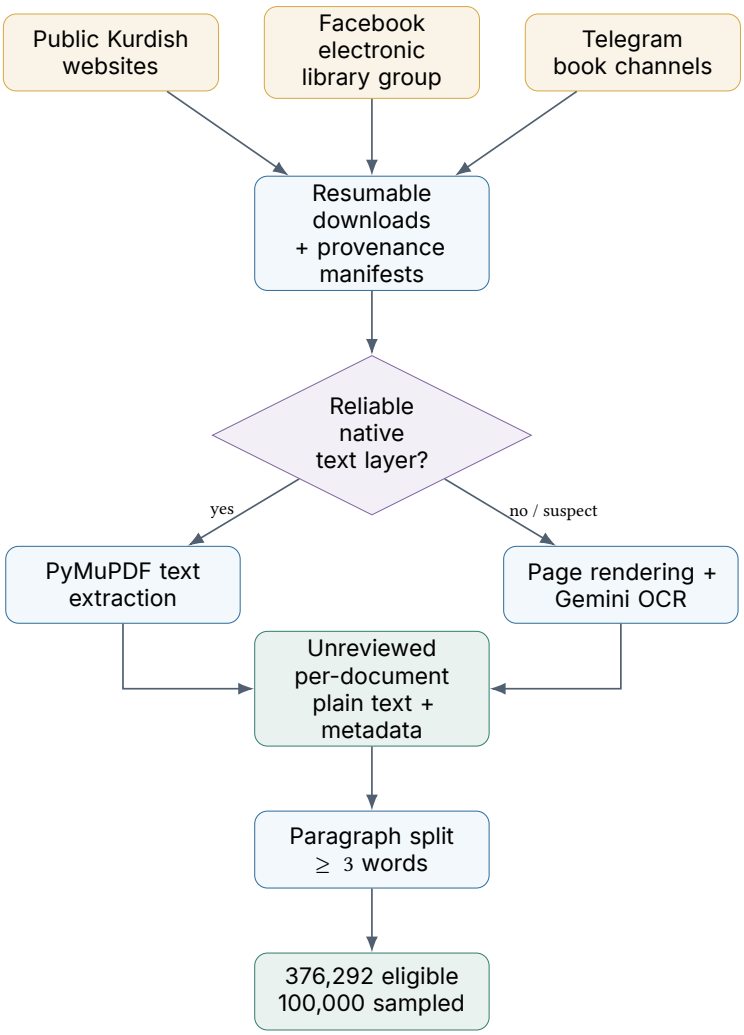


Figure 6: Verified corpus provenance from Bahdini Crawler to the deployed analytical sample. "Unreviewed" is retained as an explicit quality state.

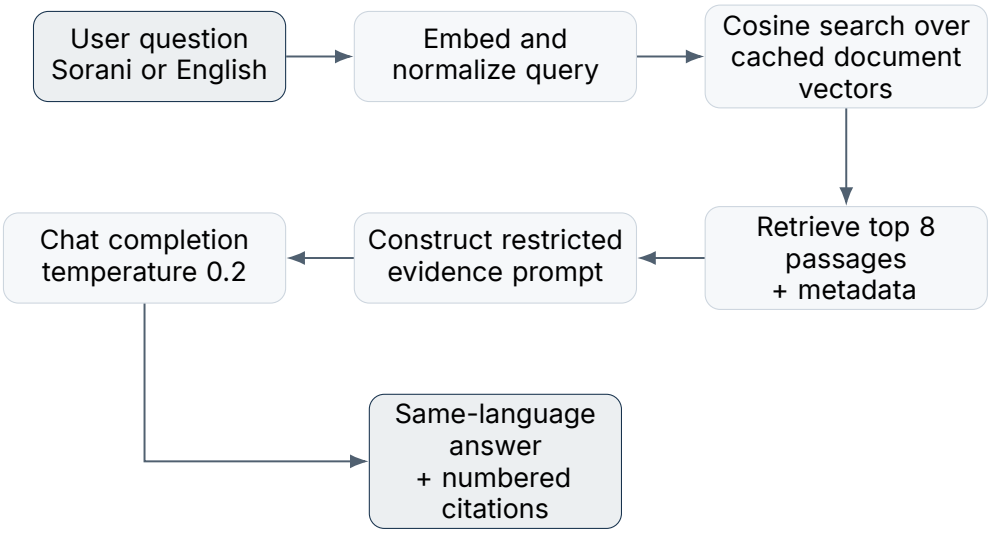


Figure 7: Grounded question-answering flow. The instruction requires the model to use only retrieved passages and to state when evidence is insufficient.

Appendix B: Live System Screenshots

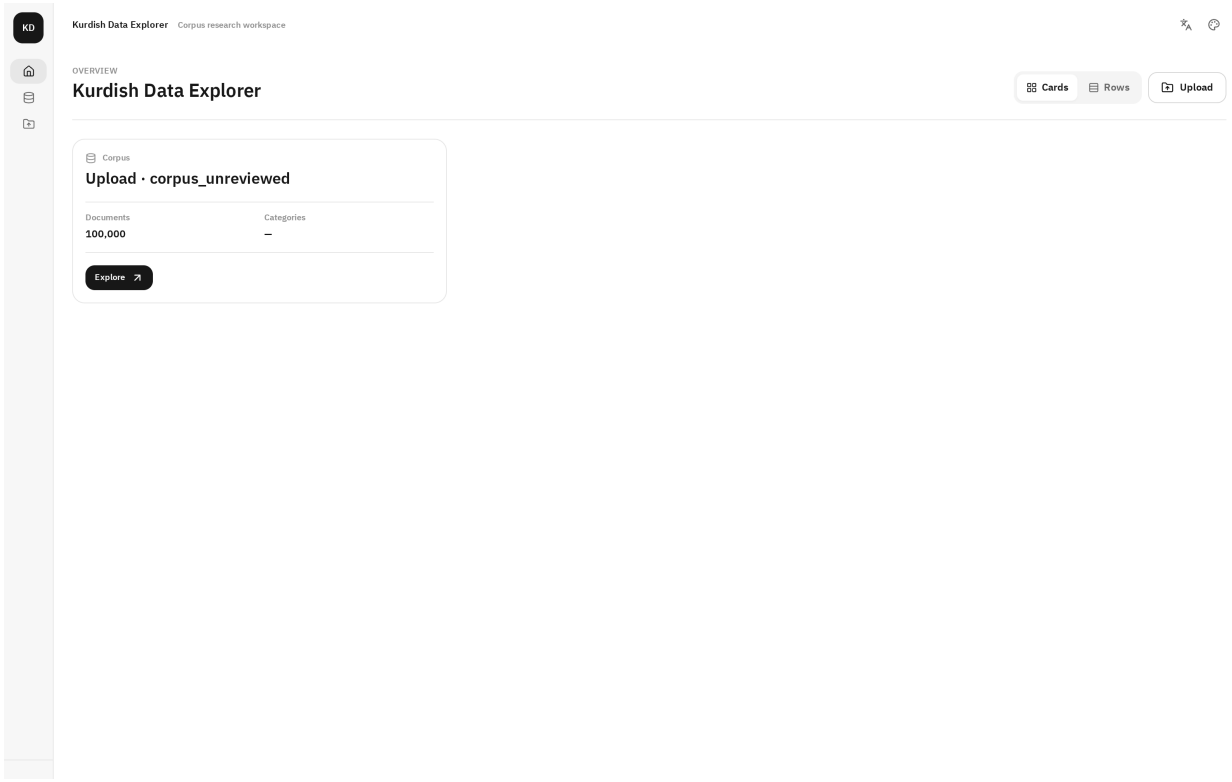


Figure 8: Corpus overview: the controlled deployment exposes one 100,000-document source.

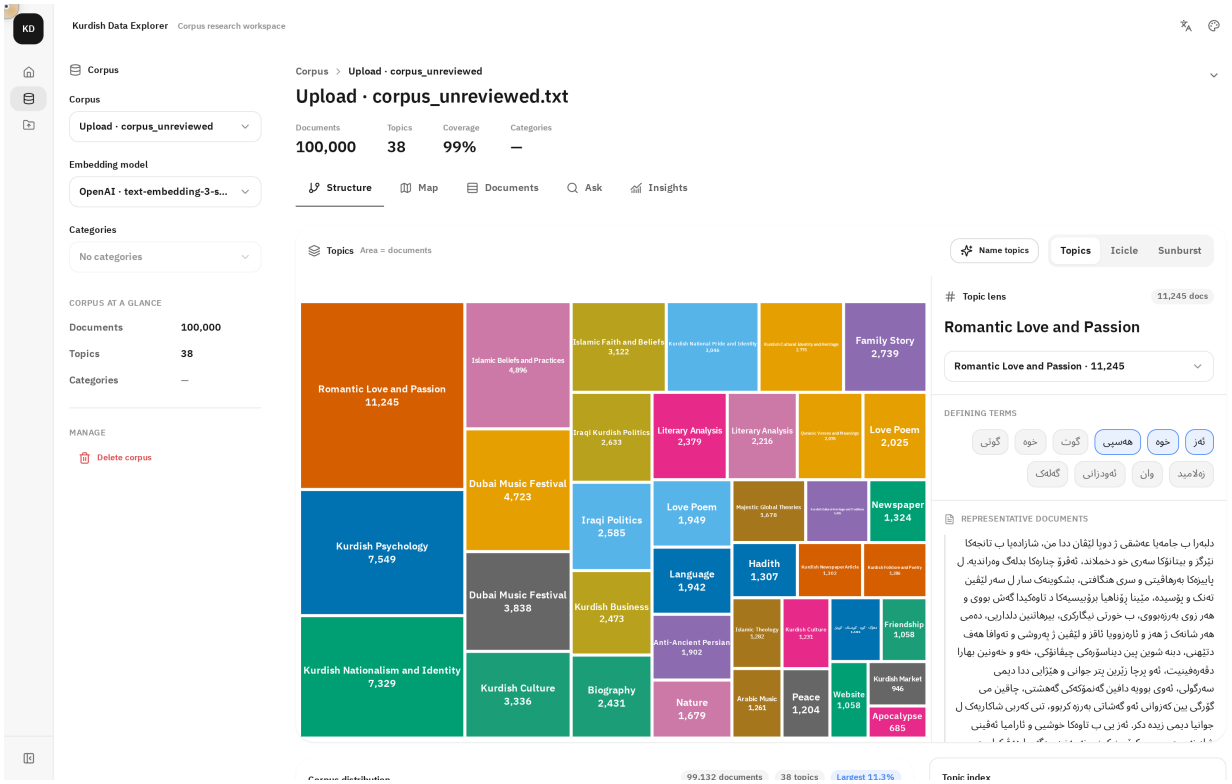


Figure 9: Structure workspace: topic treemap, defining terms, and representative Kurdish documents.

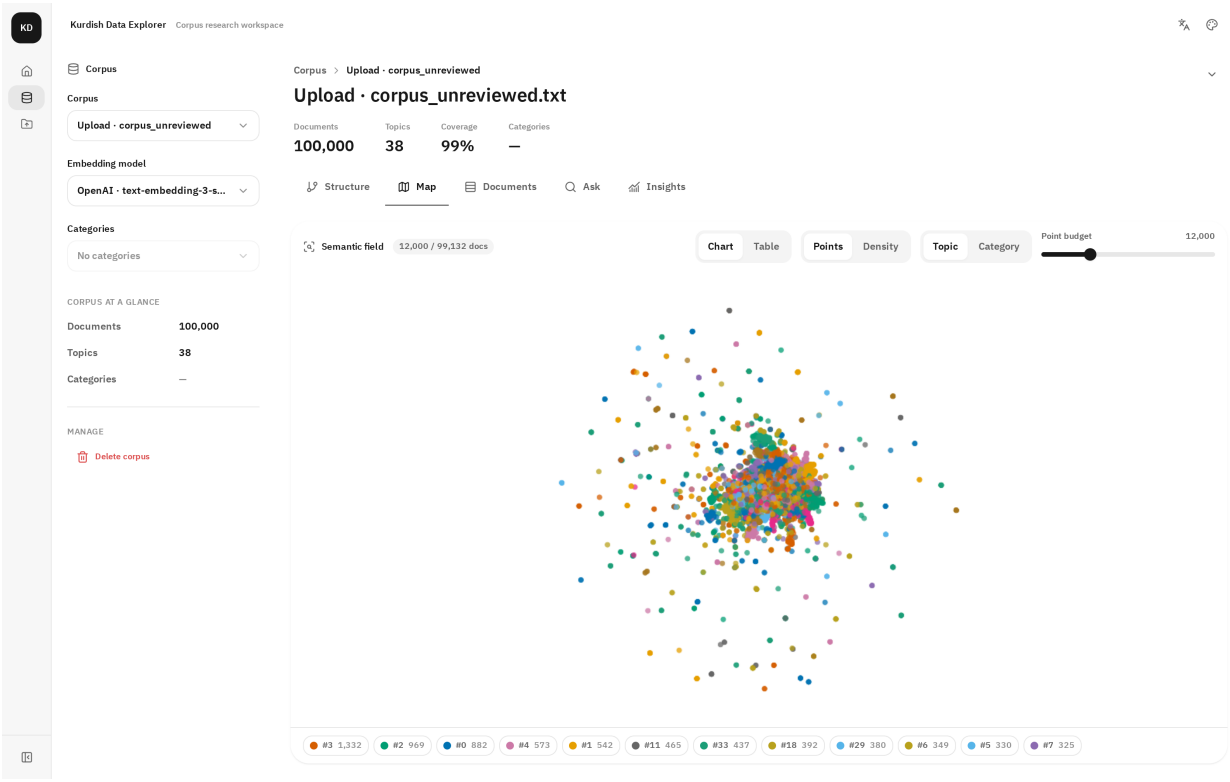


Figure 10: Map workspace: 12,000 displayed points sampled from 99,132 clustered OpenAI documents.

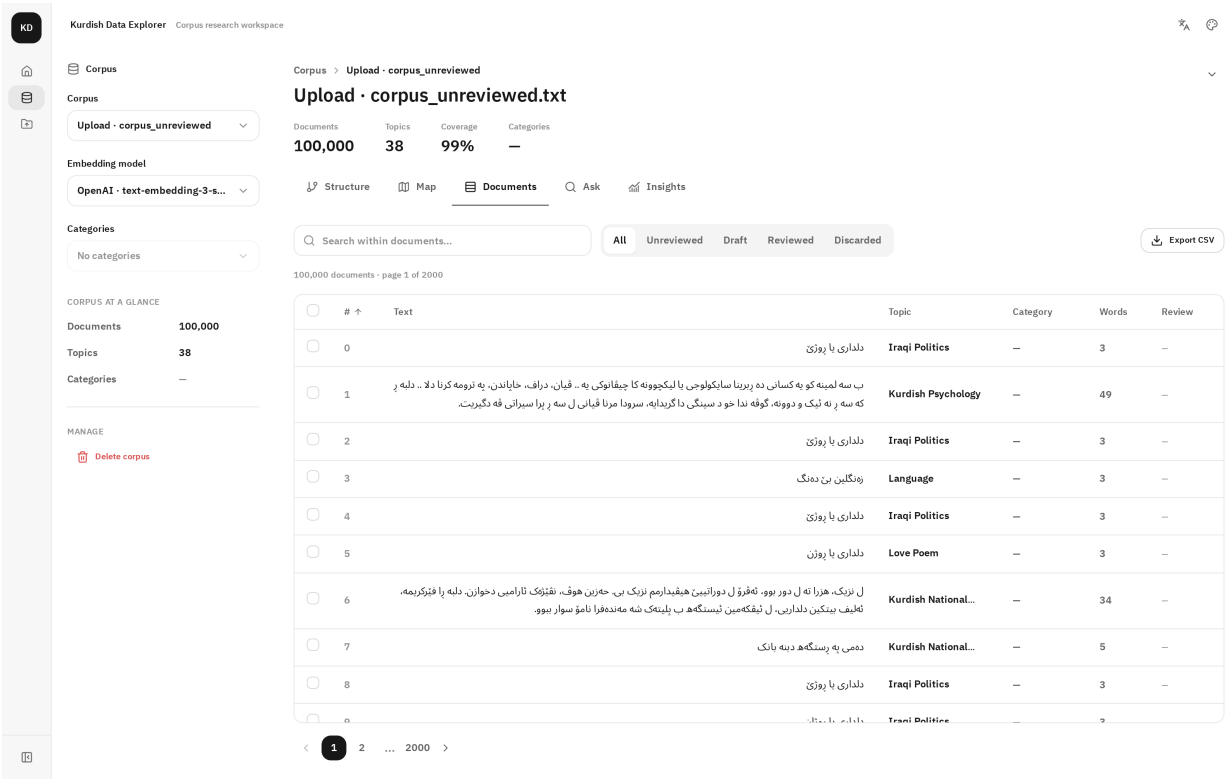


Figure 11: Documents workspace: searchable table, topic assignments, word counts, and review states.

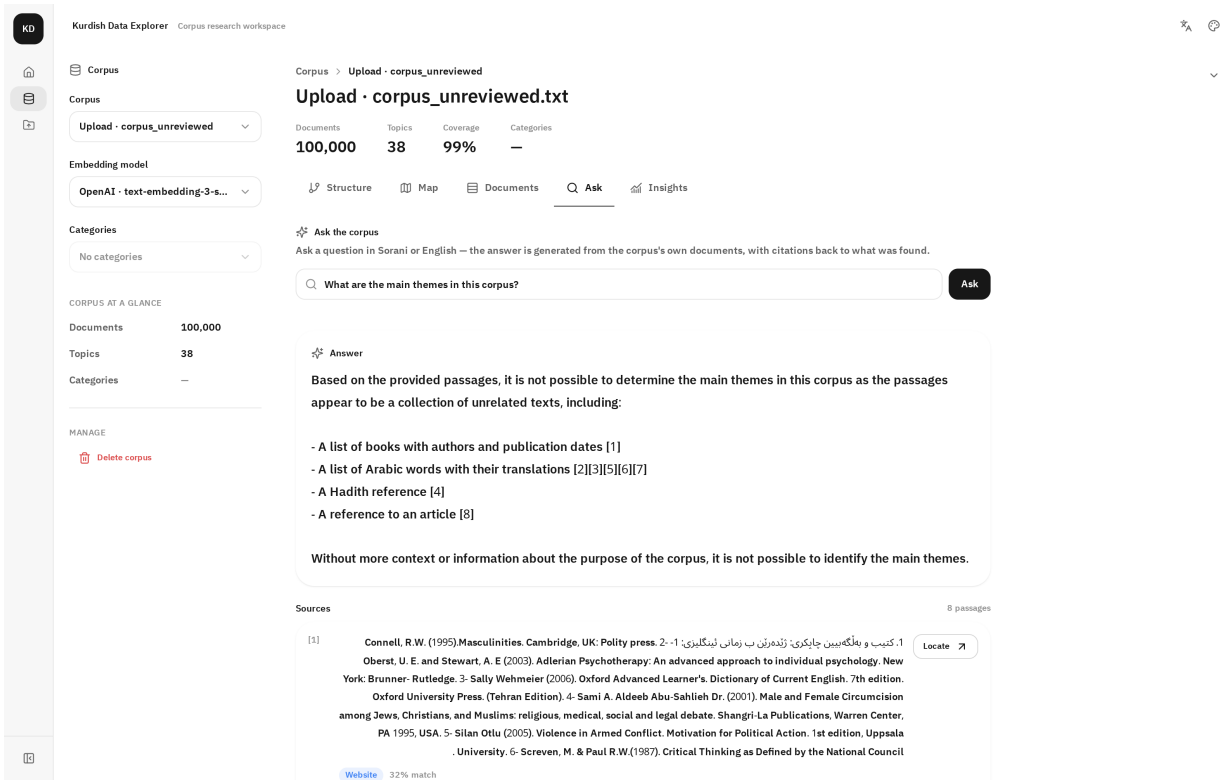


Figure 12: Sample input and output: an English question answered from eight retrieved passages. The cautious answer illustrates evidence-bounded RAG behavior.

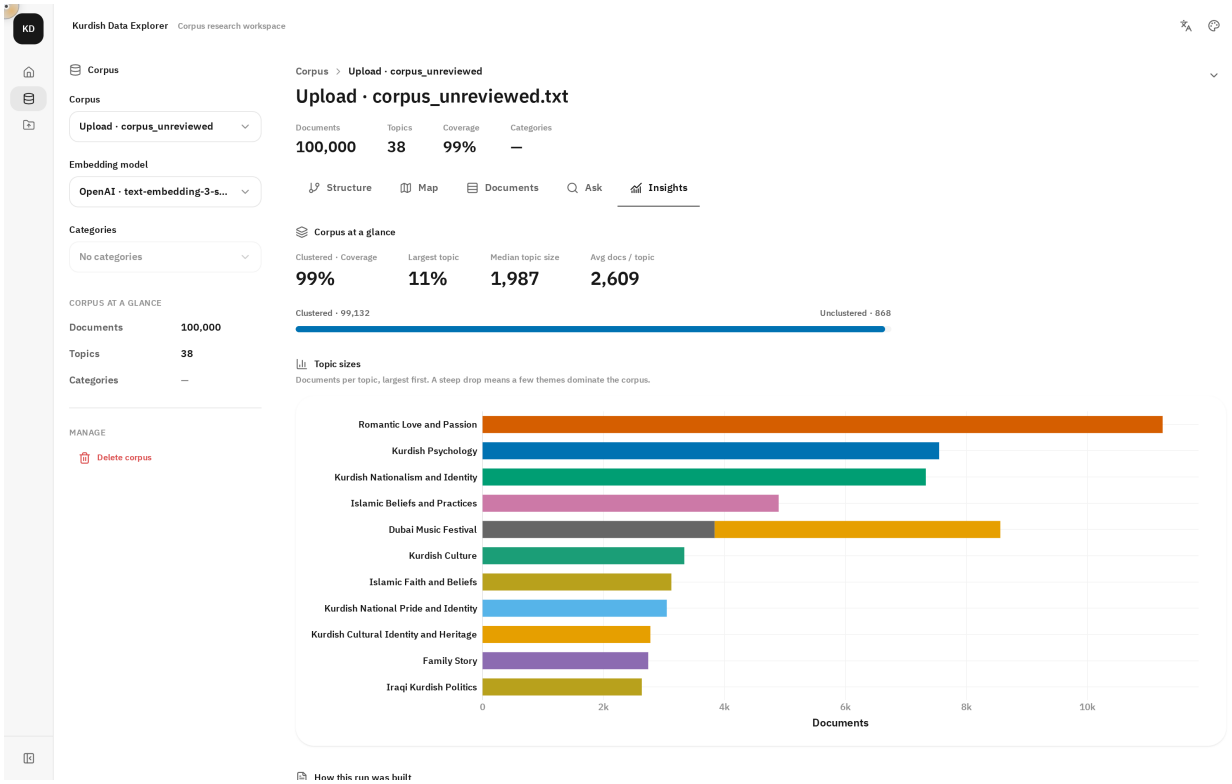


Figure 13: OpenAI run insights: topic-size distribution and exact run provenance.

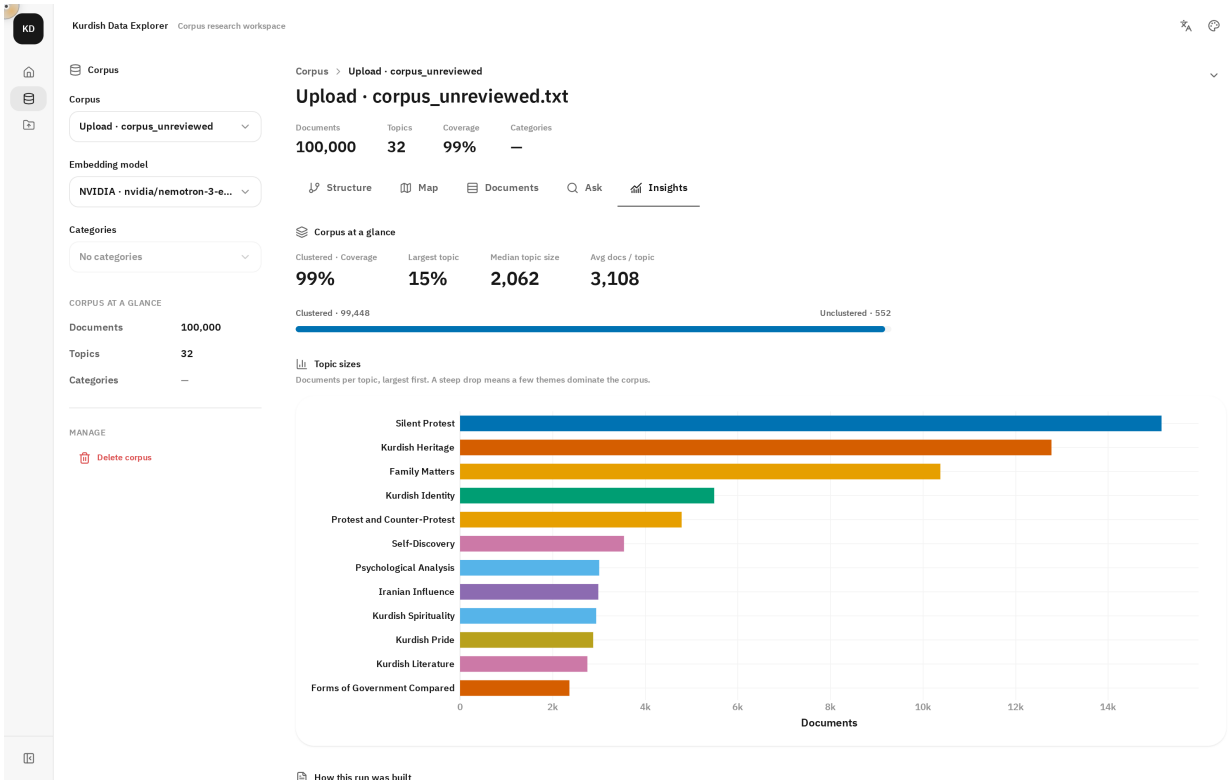


Figure 14: NVIDIA run insights after switching the model selector in the live application.

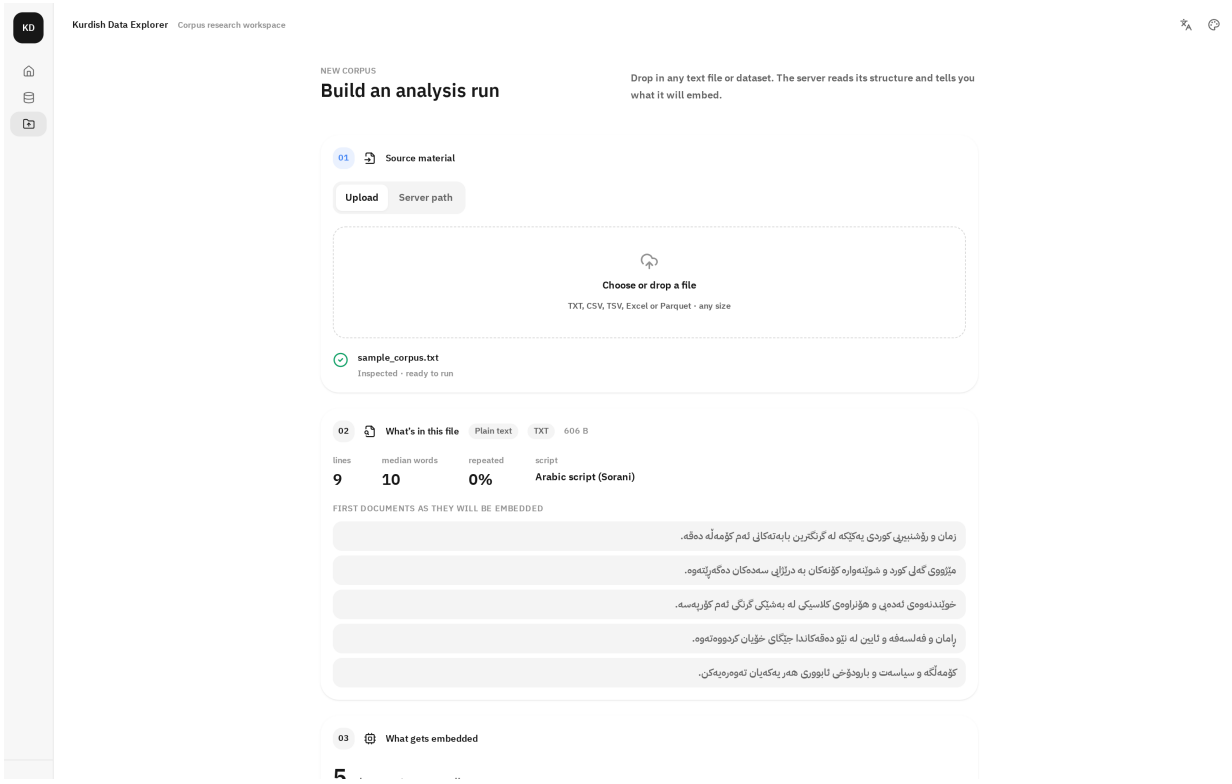


Figure 15: Upload workflow for TXT, CSV, TSV, Excel, or Parquet source material.

Appendix C: Artifact and Submission Map

Table 4: Submission requirements and the corresponding project deliverables.

Requirement	Project location or access method
Final report PDF	`reports/final/Kurdish_Data_Explorer_Final_Report.pdf`
LaTeX source	`reports/final/main.tex` and `references.bib`
Source code	github.com/SawabS/NLP_KurdishDataExplorer
Dataset instructions	github.com/SawabS/Bahdini_Crawler; raw/OCR data are excluded from Git
Trained/fitted models	`artifacts/<source>__<model>/`; production image includes OpenAI and NVIDIA run artifacts
Presentation slides	Existing project presentation materials are documented in `wiki/synthesis/project-presentation-overview.md`
Deployment	https://kdx-explorer.fly.dev; health endpoint at https://kdx-explorer.fly.dev/api/health

Kurdish Data Explorer
https://kdx-explorer.fly.dev